TECHNICAL SERVICE GUIDE

NxtGen AI Services

June 2026 deck translated into an engineering view

A practical guide to the AI infrastructure, GPU-as-a-Service, managed operations, model/runtime choices, and SAS-F delivery framework presented in the source deck.

Sovereign AI for production workloads

Use this document to understand what each service is for, where it fits in an AI stack, and what to confirm before committing to a design or proof of concept.

Prepared from: NxtGen_AI_Services_-_Deck_-_Jun2026.pdf
Important: This is a technical interpretation of the supplied deck, not an independent performance, compliance, or availability certification.

01 | EXECUTIVE READING

What the deck is saying

The June 2026 deck positions NxtGen as a sovereign-AI services provider with offerings across the AI stack. Its central message is that enterprises need more than GPU capacity: they need a governed path from infrastructure to deployed, continuously improved AI solutions.

The source deck presents four connected service areas:

- GPU-as-a-Service: consume heterogeneous GPU capacity without building the full physical platform yourself.
- AI Infrastructure Managed Services: delegate operation of storage, compute, Kubernetes, OpenShift, backup, security, networking, and migration layers.
- Full Stack AI Development by SAS-F: follow a structured path from bespoke solution engineering through deployment and continuous improvement.
- Model and runtime ecosystem: use open-source models, NVIDIA NIM, vLLM, and llm-d across the selected hardware estate.

Best fit: organizations that need production AI capability in India, want to choose from more than one GPU vendor, and prefer a managed operating model. The deck is strongest as a portfolio map; detailed sizing, service levels, and evidence still need to be confirmed for each workload.

Claim versus engineering implication

Deck claim or signal	Engineering implication
Sovereign AI	Ask where data is stored and processed, who can administer the environment, where support access originates, and which controls prove residency and operational sovereignty.
Services at every layer of the AI stack	Define the boundary between customer-owned code/data and provider-managed infrastructure, platform, security, and operations.
Heterogeneous GPU platform	Select hardware by model size, precision, memory, interconnect, concurrency, and cost per useful output - not by brand alone.
Patent pending SAS-F	Treat SAS-F as a delivery framework; request the lifecycle artifacts, acceptance gates, ownership model, and repeatability evidence during discovery.

02 | PORTFOLIO MAP

How the services fit together

The easiest way to avoid confusion is to separate the AI stack into four questions: what hardware is available, who operates it, how the solution is built, and how the model runs in production.

Service area	Primary responsibility	Typical consumer
GPU-as-a-Service	Provide GPU-backed capacity, networking, platform integrations, and a path to allocate or scale accelerators.	AI/ML teams, data scientists, model-serving teams, HPC users.
AI infrastructure managed services	Operate the supporting layers: storage, VMs, operating systems, Kubernetes, OpenShift, backup, DR, security, VPN, and migration.	Infrastructure, security, SRE, and platform engineering teams.
Full-stack AI development by SAS-F	Engineer a bespoke solution, curate and centralize data, develop the AI solution, validate guardrails, deploy it, and continuously improve it.	Business owners with a defined use case but incomplete internal AI delivery capability.
Models and runtimes	Choose model families and inference engines that match licensing, quality, latency, throughput, and hardware constraints.	ML engineers, application developers, and inference operators.

Example: an internal enterprise assistant

- GPU-as-a-Service provides the accelerator capacity for embedding, retrieval, and generation workloads.
- Managed object or parallel file storage holds source documents, indexes, model artifacts, and logs.
- OpenShift or Kubernetes packages the API, retrieval service, model server, and monitoring components.
- SAS-F structures data curation, guardrail validation, deployment, and continuous improvement.
- Managed backup, firewall, VPN, EDR, and operating-system services reduce the operational burden around the application.

03 | AI ADOPTION CONTEXT

Why infrastructure and delivery discipline matter

The deck uses adoption statistics to frame the problem: organizations are experimenting with AI, but infrastructure complexity, skills shortages, sovereignty concerns, use-case ambiguity, and cost uncertainty slow production adoption.

Signal shown in source deck	What to do technically
47% of Indian firms have multiple GenAI use cases live	Create a reusable platform baseline instead of a separate bespoke stack for every pilot: identity, network, model access, logging, data controls, and deployment templates.
54% struggle to hire skilled AI professionals	Prefer managed operations and platform guardrails for the layers that do not differentiate the business; reserve internal skills for data, domain logic, evaluation, and product outcomes.
81% feel pressure to speed up AI	Use a staged path from use-case definition to controlled pilot to production, with measurable acceptance criteria at every gate.
80% need more AI compute in the medium term	Plan capacity for both training and inference. Track GPU memory, utilization, queue time, concurrency, tokens per second, and cost per useful transaction.
Infrastructure complexity is shown as the largest barrier	Treat storage, networking, identity, observability, security, and recovery as part of the AI product architecture from the beginning.

Practical takeaway

A GPU is not an AI platform. A production AI service needs data pipelines, model/runtime packaging, network and storage performance, identity and security, observability, evaluation, recovery, and a clear owner for each layer.

Use the adoption statistics as business context, not as workload sizing. For sizing, replace broad percentages with your own model family, input/output token profile, concurrency, latency target, data volume, retention period, and recovery objectives.

04 | GPU-AS-A-SERVICE

Choosing the accelerator path

The deck presents a heterogeneous GPU platform and lists NVIDIA, AMD, and Intel options. The right choice depends on model memory, precision, interconnect, inference concurrency, training scale, framework support, and operational constraints.

Hardware or platform signal	Source details	Where it may fit
NVIDIA RTX PRO 6000 Blackwell	8 GPUs, 96 GB each; 200 Gbps network through two 100GbE switches; Red Hat OpenShift.	Media, graphics, workstation-style AI, computer vision, and inference that benefit from strong GPU memory and multimedia performance.
NVIDIA H200NVL	8 GPUs, 141 GB each; 200 Gbps network; Red Hat OpenShift and NVIDIA Enterprise AI Suite.	General AI applications, larger models, high-throughput inference, and workloads that need more accelerator memory.
Heterogeneous catalog	The deck also shows NVIDIA H200, H100, B200, AMD MI325x, and Intel Gaudi 2.	Compare performance, framework compatibility, availability, price, and support model before standardizing on one accelerator family.

How to size a GPU service

- Model memory: include weights, KV cache, activations, framework overhead, batching, and headroom - not just the nominal parameter count.
- Training versus inference: training emphasizes memory bandwidth, interconnect, checkpointing, and distributed scaling; inference emphasizes latency, concurrency, batching, and utilization.
- Network: confirm whether the stated 200 Gbps is a host, fabric, aggregate, or effective application throughput figure, and test sustained traffic under the intended topology.
- Software: verify driver, CUDA/ROCm/oneAPI, container, OpenShift/Kubernetes, model-server, and monitoring compatibility for the exact versions you will operate.
- Commercial unit: compare cost per GPU-hour and cost per useful output, not only the hourly price of a node.

Example: document classification at scale

For a document-classification pipeline, start with the smallest accelerator that meets the batch throughput target. Benchmark preprocessing, inference, post-processing, storage reads, and queue time together. A larger GPU is useful only if the end-to-end pipeline can feed it and the business benefits from the reduced completion time.

05 | MODELS AND RUNTIMES

Open models, inference engines, and the serving contract

The deck shows Qwen, Llama, DeepSeek, and NVIDIA Nemotron as model examples, and NVIDIA NIM, vLLM, and llm-d as runtime or serving components. These are building blocks, not a complete application architecture.

Layer	Examples shown in deck	Questions to answer
Model family	Qwen, Llama, DeepSeek, NVIDIA Nemotron.	What quality, language coverage, license, context length, safety behavior, and fine-tuning options are required?
Inference/runtime	NVIDIA NIM, vLLM, llm-d.	Which runtime supports the target GPU, quantization, batching, parallelism, metrics, rollout, and failure behavior?
Platform	Red Hat OpenShift and the GPU platform shown in the deck.	How are namespaces, quotas, secrets, images, networking, storage, and upgrades governed?
Application	Not specified by the deck; supplied by the customer or SAS-F delivery team.	What APIs, prompts, tools, retrieval, user permissions, evaluations, and business workflows surround the model?

Serving example: a retrieval-augmented assistant

- Ingest and curate documents into a governed object or file-storage zone.
- Generate embeddings and build a vector index; keep source references so responses can be audited.
- Serve the language model through the selected runtime with authentication, rate limits, batching, and model metrics.
- Apply retrieval filters and authorization before context reaches the model.
- Measure answer quality, citation accuracy, latency, token usage, refusal behavior, and cost per request.
- Version the model, prompt, retrieval configuration, evaluation set, and deployment image together.

The model name is not the architecture. The production contract is the combination of model version, runtime version, hardware profile, prompts, retrieval data, guardrails, observability, and rollback procedure.

06 | MANAGED AI INFRASTRUCTURE

What each managed service is for

The source deck lists a broad managed-services catalog. The bullets below translate each item into an operational responsibility and a practical reason to include it in an AI platform design.

Service	Practical meaning and example
Managed object storage	Durable bucket-style storage for datasets, model artifacts, logs, and backups. Example: store immutable training data and versioned evaluation sets.
Managed parallel file system	Shared high-throughput file access for distributed training or preprocessing. Example: multiple workers read the same large corpus without copying it to every node.
Virtual machine and OS management	Provisioning, patching, health, and lifecycle operations for VM-based services. Example: keep a database, bastion, or integration tier maintained while the AI team owns the application.
Kubernetes and OpenShift management	Operate the cluster and platform baseline. Example: run model servers, APIs, jobs, secrets, quotas, and rolling deployments with consistent controls.
DR-as-a-Service and backup	Recover data and services after failure or operational error. Example: restore model artifacts, indexes, configurations, and application state against defined RPO/RTO targets.
NVIDIA Base Command Manager	Manage an NVIDIA-oriented AI/HPC cluster workflow. Confirm supported features, version, tenancy, and the division of responsibilities.
EDR, firewall, and VPN	Protect endpoints, network paths, and administrative access. Example: isolate a GPU environment from public ingress while allowing approved users and pipelines to reach it.
Managed migration	Move workloads or data with planning, dependency mapping, validation, and cutover support. Example: migrate an AI pilot from a lab cluster to managed OpenShift with a rollback plan.

The key design question

Which layers must your team operate directly, and which layers should the managed-service provider operate under agreed service levels? Put this boundary in writing before a proof of concept becomes production dependency.

07 | SAS-F DELIVERY FRAMEWORK

From bespoke idea to continuously improved service

The Standardized AI Solutions Framework (SAS-F) is described in the deck as a structured approach to engineering and delivering AI services. Its visual shows five stages that can be used as a practical delivery checklist.

SAS-F stage	Technical work	Exit evidence
Bespoke solution engineering	Define the business outcome, users, data boundaries, interfaces, constraints, and non-functional requirements.	Approved problem statement, architecture outline, success metrics, owners, and risk register.
Data curation and centralization	Inventory sources, establish quality rules, classify sensitive data, create ingestion and transformation flows, and version datasets.	Lineage, data-quality report, access policy, retention rules, and reproducible dataset or index build.
AI solution development	Select model and runtime, design prompts or fine-tuning, implement retrieval/tools, package services, and create evaluation sets.	Versioned model/application build with offline evaluation results and resource profile.
Guardrailing, evaluation, and validation	Test quality, safety, leakage, bias, prompt injection, access control, performance, failure modes, and cost.	Acceptance report, red-team findings, remediation status, and production go/no-go decision.
Deploy, manage, and continuously improve	Release through controlled environments, monitor service health and model quality, manage drift, update versions, and roll back safely.	Runbook, dashboards, SLOs, incident process, change history, and improvement backlog.

Example: claims-document triage

- Define: reduce manual triage time while keeping a human approval step for high-risk claims.
- Curate: centralize policy documents and historical claims with masking and role-based access.
- Develop: use retrieval plus a classification model; produce a reason code and source references.
- Validate: measure precision/recall by claim type, false-positive cost, leakage, and response latency.
- Operate: monitor drift in document types and approval outcomes; retrain or revise the workflow when quality declines.

08 | REFERENCE ARCHITECTURE

A production pattern assembled from the deck

The following pattern is an example, not a prescriptive NxtGen reference architecture. It shows how the listed capabilities can be composed into a governed AI service.

Flow	Example components	Control points
Users and applications	Web app, API clients, workflow systems, internal portals.	Identity, authorization, rate limits, tenant isolation, audit events.
AI service layer	Prompt/orchestration service, retrieval service, tool connectors, model gateway.	Prompt versioning, input/output filtering, model allow-list, trace IDs, timeouts.
Model serving	NVIDIA NIM, vLLM, or llm-d on the selected GPU platform.	GPU quotas, batching, concurrency, latency, tokens/sec, health checks, canary and rollback.
Data and artifacts	Object storage, parallel file system, vector index, model registry, evaluation sets.	Classification, encryption, lineage, retention, backup, restore test, access review.
Platform and operations	OpenShift/Kubernetes, VM and OS management, firewall, VPN, EDR, DR, monitoring.	SLOs, patching, vulnerability management, incident response, capacity and cost reporting.

Minimum production checklist

- A named owner for data, model, application, platform, security, and service operations.
- A measurable quality target and a latency/throughput target for the actual user journey.
- A repeatable build path for model, prompt, data, image, configuration, and infrastructure versions.
- A recovery test covering data, indexes, models, secrets, configuration, and deployment instructions.
- A cost view that separates GPU, storage, network, platform, support, and engineering effort.

09 | VALIDATION AND GOVERNANCE

Questions to ask before selecting a service

The deck establishes the portfolio; a technical evaluation should convert it into measurable acceptance criteria. Use the questions below in a discovery workshop or proof-of-concept plan.

Area	Questions to validate
Sovereignty and access	Where are data, backups, logs, and support artifacts stored? Who can access them? Are administrator actions logged? How is cross-border support or remote access handled?
Capacity and performance	What is reserved versus shared? What are the GPU memory, interconnect, storage IOPS, bandwidth, latency, queue time, and availability characteristics under load?
Software compatibility	Which versions of OpenShift/Kubernetes, drivers, runtimes, model servers, registries, storage clients, and observability tools are supported? How are upgrades tested?
Security	How are identity, MFA, secrets, network segmentation, encryption, EDR, vulnerability remediation, and incident response implemented and evidenced?
Reliability and recovery	What are the service levels, maintenance windows, RPO/RTO, backup retention, DR topology, restore procedure, and test frequency?
Commercial model	Are charges based on GPU-hours, reserved capacity, storage, egress, managed services, support tier, or minimum commitment? What happens during idle periods?
SAS-F delivery	What artifacts are produced at each stage? Who owns the data and model assets? What constitutes acceptance, and what is included in continuous improvement?

A useful proof of concept measures the complete path: data read -> preprocessing -> model execution -> post-processing -> response delivery -> logging. Benchmarking only the GPU kernel or a synthetic prompt can hide the real bottleneck.

10 | EXAMPLE WORKLOAD DECISIONS

Which service combination fits which need?

The same AI platform can support very different workloads. Start with the workload shape, then select the smallest set of services that satisfies performance, security, and operational requirements.

Workload	Likely service combination	Key measurement
Interactive GenAI assistant	GPU-as-a-Service + model runtime + object storage + OpenShift/Kubernetes + firewall/VPN + evaluation and monitoring.	p95 latency, time to first token, tokens/sec, answer quality, citation accuracy, cost/request.
Large-scale model training or fine-tuning	Heterogeneous GPU platform + parallel file system + high-throughput network + cluster management + checkpoint backup.	samples/sec, scaling efficiency, GPU utilization, checkpoint time, job completion time, failure recovery.
Computer vision or media analytics	RTX PRO 6000-class option or other suitable GPU + object storage + batch orchestration + managed OS/cluster operations.	frames/sec, batch throughput, model accuracy, decode/preprocess time, queue time, cost/video hour.
Regulated enterprise AI service	SAS-F + managed platform + private networking/VPN + EDR/firewall + backup/DR + documented data and access controls.	control evidence, auditability, recovery objectives, data leakage rate, approval time, service availability.
Partner-delivered AI platform	GPU service + multi-tenant OpenShift/Kubernetes + provisioning automation + quotas + billing/usage boundaries + managed security.	tenant isolation, provisioning time, noisy-neighbor impact, usage accuracy, support response, gross margin.

Decision shortcut

- Need capacity only? Start with GPU-as-a-Service and benchmark the model/runtime combination.
- Need a production platform? Add storage, cluster, security, backup, monitoring, and operating services.
- Need an outcome but not a complete internal AI team? Use SAS-F to structure delivery and ownership.
- Need a customer-facing service? Add tenancy, provisioning, quota, usage, support, and commercial controls.

11 | QUICK REFERENCE

A practical starting plan

Use this sequence to move from the supplied portfolio description to a defensible technical decision.

Step	Output
1. Define the workload	Use case, users, data classes, model/task, input/output profile, accuracy target, latency target, throughput, and business value.
2. Classify the operating need	Sovereignty, security, connectivity, tenancy, availability, RPO/RTO, and support requirements.
3. Select an initial stack	GPU option, model/runtime, storage, OpenShift/Kubernetes or VM pattern, networking, and managed-service boundary.
4. Build a representative POC	End-to-end workload with production-like data shape, security controls, observability, and failure scenarios.
5. Validate and govern	Quality, safety, performance, cost, data lineage, recovery, and operational acceptance evidence.
6. Plan scale and improvement	Capacity plan, upgrade path, model refresh process, drift monitoring, support model, and continuous-improvement backlog.

Bottom line

The NxtGen AI Services deck describes a full-stack path: sovereign GPU capacity, a heterogeneous hardware and runtime ecosystem, managed AI infrastructure, and a structured delivery framework. The strongest technical evaluation will connect those pieces to one measurable workload and test the entire operating model - not just the accelerator.

Source boundary

All product names, hardware configurations, service names, adoption statistics, and SAS-F descriptions in this guide are derived from the supplied June 2026 deck. They should be confirmed against current service catalogs, contracts, architecture diagrams, security documentation, benchmark methodology, and availability by region before procurement or production use.

Contact shown in source deck: marketing@nxtgen.com